Final Report: Prompt Engineering Lab (M2.01)

Failure Analysis Summary (15-run consistency)

Metrics

- **Format compliance %**: outputs match the required production format
  - Sentiment: single word in {positive, negative, neutral}
  - Product: one paragraph, 3–4 sentences
  - Extraction: exactly 2 lines, starting with Main issue: and Overall sentiment:
- **Most-common output %**: outputs are exactly identical to the most frequent response

Task	Version	Format compliance %	Most-common output %
Sentiment	v1	0%	73.3%
Sentiment	v3 (few-shot)	100%	100%
Product description	v1	60%	26.7%
Product description	v3 (few-shot)	100%	33.3%
Data extraction	v1	0%	80%
Data extraction	v3 (CoT)	100%	100%

Failure Patterns Observed (v1)

- **Sentiment analysis**
  - Returned explanations instead of a single label
  - Practical impact: breaks pipelines and dashboards that require strict labels
- **Product description**
  - Sentence count drifted (3–5 sentences), tone and closings varied
  - Practical impact: inconsistent brand voice and unstable UI rendering
- **Data extraction**
  - Output used bullets/markdown and varied phrasing instead of a stable schema
  - Practical impact: brittle parsing and higher cleanup effort

v1 → v3 Improvements

- **Sentiment (v3 few-shot):** moved from verbose explanations and label drift to single-label outputs with perfect format compliance.
    - Sometimes produced **"Mixed"**, which was not an allowed class
- **Product (v3 few-shot):** enforced paragraph + sentence constraints; style became consistent while allowing some controlled wording variation.
- **Extraction (v3 CoT):** standardized to a strict two-line output with fully stable phrasing across runs.

Improved Prompts (Final v3)

Sentiment prompt (v3, few-shot)

You are a sentiment classification system.

Task:
Classify the sentiment of the customer message below.

Rules:
- Choose exactly one of the following labels: positive, negative, or neutral
- Return only the label
- Do not include explanations, punctuation, or additional text

Examples:

Customer message:
"The support team was very helpful and solved my issue quickly."
Sentiment:
positive

Customer message:
"The app works fine, but there is nothing special about it."
Sentiment:
neutral

Customer message:

"I am very disappointed because the app keeps crashing."
Sentiment:
negative

Customer message:
"I love the new design of your website, but the checkout process is really frustrating."

Sentiment:

Product prompt (v3, few-shot)

You are a marketing copy generator.

Task:
Write a product description using the information below.

Structure:
- One paragraph only
- 3–4 sentences total

Style guidelines:
- Professional, clear, and confident tone
- No exaggerated claims or invented features
- Focus on practical benefits for busy professionals

Constraints:
- Use only the provided product information
- Do not add bullet points, headings, or emojis

Examples:

Product information:
Product name: AeroDesk Laptop Stand
Features: Adjustable height, aluminum body, foldable design
Target audience: Remote workers

Product description:

The AeroDesk Laptop Stand is designed for remote workers who need a flexible and ergonomic workspace. Its adjustable height and sturdy aluminum body provide comfort and stability throughout the workday. The foldable design makes it easy to store or carry between workspaces, supporting productivity wherever you work.

Product information:
Product name: SmartFlow Water Bottle
Features: Keeps drinks cold for 24 hours, stainless steel body, leak-proof lid
Target audience: Busy professionals

Product description:

Extraction prompt (v3, CoT)

You are an information extraction system.

Task:
Extract specific fields from the customer feedback below.

Fields to extract:
- Main issue: the primary problem described by the customer
- Overall sentiment: one short phrase describing the sentiment

Reasoning:
First, think step by step to identify the issue and sentiment. Keep your reasoning private and do not output it.

Format requirements:
- Use exactly two lines
- Start each line with the field name followed by a colon
- Do not add explanations or additional text
- Output must be plain text (no bullet points, no markdown)

Customer feedback:
"I love the design of the app, but it crashes every time I try to upload a file."

Why these changes worked

- **Sentiment:** hard constraints plus few-shot examples reduced explanation drift and prevented unsupported labels.
- **Product:** structure + style requirements stabilized length and tone while keeping natural variation.
- **Extraction:** schema-first formatting plus hidden CoT produced consistent, parse-friendly output without reasoning leakage.

Reflection

Systematic testing

Running each prompt 5, 10, and then 15 times exposed issues that a single test completely hid. With one run, the outputs looked "good enough," but repetition showed clear patterns: format drift, unsupported labels like *"Mixed"*, and inconsistent verbosity. The content was usually correct, yet the way it was delivered was unreliable. This made it obvious that prompt quality has to be evaluated statistically, not anecdotally. Consistency only became measurable once the prompts were stressed through repetition.

Iterative improvement

The biggest gains in consistency came from tightening constraints and structure rather than changing wording. The step from v1 to v2 improved clarity, but the real jump happened in v3: adding few-shot examples for sentiment and strict schemas (plus CoT) for extraction. Few-shot examples strongly anchored the model's behavior and eliminated most ambiguity, while schema-first formatting removed parsing issues. Iteration showed that small, targeted changes can outperform large prompt rewrites.

Technique selection

Few-shot prompting works best for classification and generation tasks where the desired behavior can be demonstrated clearly through examples. It is especially effective when the output space is small and well-defined. Chain-of-Thought is most useful for analysis and extraction tasks that require reasoning over messy or complex input. In those cases, keeping the reasoning internal while enforcing a strict output format delivers both accuracy and reliability. The choice depends on task complexity, output strictness, and whether reasoning needs to be exposed or hidden.

Real-world application

In a production AI system, I would treat prompts as versioned components and validate them through repeated testing before deployment. Classification tasks would use closed label sets with few-shot examples and hard output constraints. Generative tasks would balance structure with controlled flexibility to maintain brand consistency. Extraction tasks would rely on strict schemas and hidden CoT to ensure parseable output. Most importantly, I would continuously test prompts against input variations and monitor consistency metrics, treating prompt engineering as an ongoing reliability exercise rather than a one-time setup.